

MACHINE LEARNING ALGORITHMS LABORATORY

Experiment 4

Binary Classification using Linear and Kernel-Based Models

Experiment No.	4
Experiment Title	Binary Classification using Linear and Kernel-Based Models
Student Name	Harshita B
Register Number	3122247001021
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	16th August 2026

Subject Code: ICS1512
Machine Learning Algorithms Laboratory

1 Objective

The objectives of this experiment are:

1. To implement binary classification using Logistic Regression and Support Vector Machine (SVM).
2. To classify email samples into spam and ham classes using the Spambase dataset.
3. To study the effect of regularization and other hyperparameters on Logistic Regression performance.
4. To compare different SVM kernels including linear, polynomial, RBF and sigmoid kernels.
5. To perform hyperparameter tuning using cross-validation.
6. To evaluate the models using accuracy, precision, recall and F1-score.
7. To compare Logistic Regression and SVM using five-fold cross-validation.
8. To analyse the effect of model selection and hyperparameter tuning on classification performance.

2 Problem Statement

The objective of this experiment is to develop binary classification models for identifying emails as either spam or ham. Logistic Regression is used as a linear probabilistic classifier, while Support Vector Machine is used as a margin-based classifier.

The performance of the models is evaluated before and after hyperparameter tuning. Different SVM kernels are also compared to understand their effect on classification performance.

The final models are evaluated using standard classification metrics and five-fold cross-validation.

3 Dataset Description

The Spambase dataset contains numerical features extracted from email content along with a binary target indicating whether an email is spam or non-spam (ham).

Property	Description
Dataset Name	Spambase Dataset
Dataset Type	Binary Classification
Application	Spam and Ham Email Classification
Target Variable	Spam / Ham
Feature Type	Numerical features extracted from email content
Number of Classes	2
Dataset Source	Spambase dataset

Note: The exact sample count, feature count and missing-value count should be entered from the dataset inspection performed in the notebook.

4 Exploratory Data Analysis

Exploratory Data Analysis was performed before training the classification models. The main purpose of EDA was to understand the structure of the dataset and identify potential preprocessing requirements.

The following aspects were considered:

- Dataset dimensions
- Data types
- Missing values
- Statistical summary
- Class distribution
- Feature distributions
- Correlation between features

4.1 Dataset Overview

The dataset was inspected to determine the number of samples, features, data types and target classes.

Observation:

The dataset consists of numerical email-related features and a binary target variable representing spam and ham emails. Understanding the dataset structure is important before applying classification algorithms.

4.2 Class Distribution

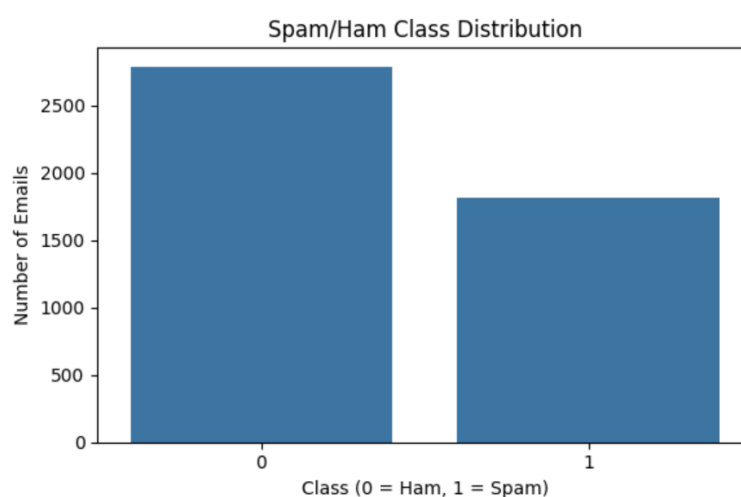


Figure 1: Distribution of Spam and Ham Emails

Observation:

The class distribution graph shows the number of spam and ham samples in the dataset. The distribution helps determine whether the dataset contains a significant class imbalance. This information is useful when interpreting accuracy along with precision, recall and F1-score.

4.3 Correlation Matrix

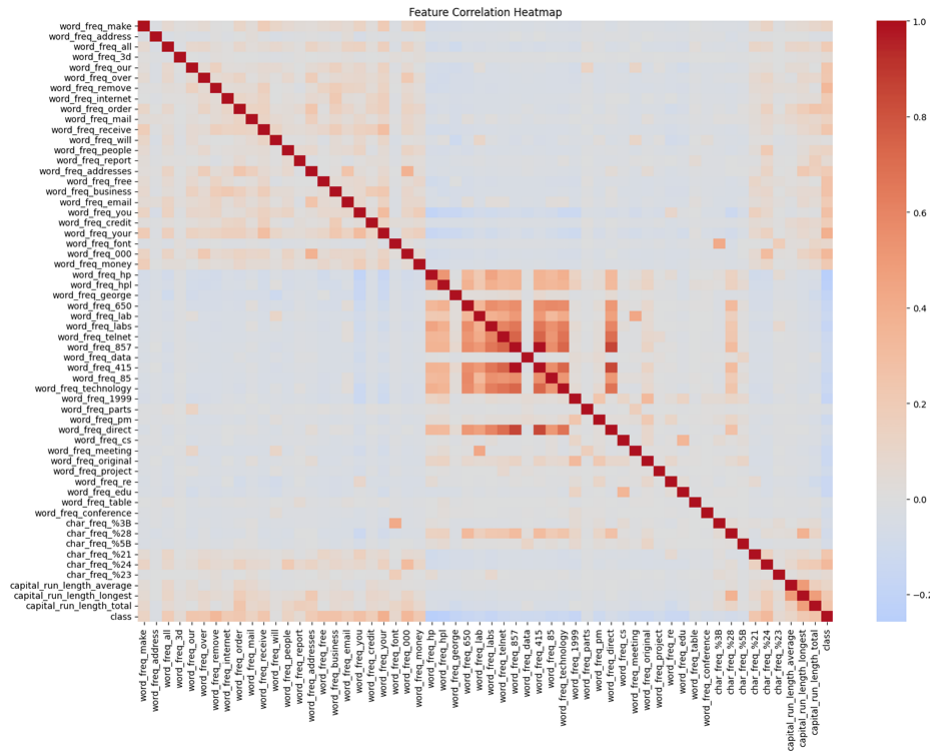


Figure 2: Correlation Matrix of Dataset Features

Observation:

The correlation matrix represents the pairwise relationship between the numerical features. Strong positive or negative correlations can indicate that some features contain related information. The matrix also provides an overview of the relationships among the input variables before model training.

5 Data Preprocessing

The following preprocessing steps were performed:

1. Missing values were checked in the dataset.
2. The target variable was separated from the input features.
3. The data was divided into training and testing sets.
4. Numerical features were standardized before model training.

5. The same preprocessing procedure was applied consistently during model evaluation.

Feature standardization is particularly important for Logistic Regression and SVM because both models are affected by the scale of the input features.

6 Machine Learning Algorithms

6.1 Logistic Regression

Logistic Regression is a probabilistic classification algorithm used for binary classification. The probability of the positive class is calculated using the sigmoid function:

$$P(y = 1|x) = \frac{1}{1 + e^{-(w^T x + b)}}$$

where w represents the model weights, x represents the input features and b is the bias term.

A threshold is applied to convert the predicted probability into a class label.

The model minimizes logistic or cross-entropy loss during training.

6.1.1 Regularization

Regularization is used to control model complexity and reduce overfitting.

Two commonly used regularization techniques are:

- **L1 Regularization:** Encourages some coefficients to become zero and can therefore perform feature selection.
- **L2 Regularization:** Penalizes large coefficients and generally provides smoother model weights.

The parameter C controls the inverse strength of regularization.

A smaller value of C results in stronger regularization, whereas a larger value allows the model to fit the training data more closely.

6.2 Support Vector Machine

Support Vector Machine is a margin-based classification algorithm. It attempts to find an optimal decision boundary that separates the classes while maximizing the margin between them.

Only a subset of training samples, known as support vectors, directly determines the decision boundary.

The general SVM optimization objective can be represented as:

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i$$

subject to:

$$y_i(w^T x_i + b) \geq 1 - \xi_i$$

where C controls the trade-off between maximizing the margin and allowing classification errors.

6.3 SVM Kernels

Different kernels were evaluated in the experiment.

- **Linear Kernel:** Suitable when the classes can be separated approximately using a linear decision boundary.
- **Polynomial Kernel:** Captures polynomial relationships between features.
- **RBF Kernel:** Can model complex non-linear decision boundaries.
- **Sigmoid Kernel:** Produces a decision function with behaviour similar to a sigmoid activation.

7 Experimental Setup

Parameter	Value
Programming Language	Python
Machine Learning Library	Scikit-learn
Dataset	Spambase
Classification Type	Binary Classification
Validation Method	Five-Fold Cross-Validation
Models	Logistic Regression and SVM

8 Hyperparameter Selection

Hyperparameter tuning was performed to identify suitable configurations for Logistic Regression and SVM.

8.1 Logistic Regression Hyperparameter Tuning

The parameters considered for Logistic Regression included:

- Regularization: L1 and L2
- C : 0.01, 0.1, 1, 10 and 100
- Solver: liblinear and saga

The best configuration obtained was:

Parameter	Value
C	1
Penalty	L2
Solver	saga

Best CV Accuracy	0.9242
Tuning Time	289.1025 seconds

Observation:

The best Logistic Regression configuration was obtained with $C = 1$, L2 regularization and the saga solver. The best cross-validation accuracy during tuning was 0.9242. The tuning process required approximately 289.10 seconds. The selected L2 regularization provides a balance between model complexity and generalization.

8.2 SVM Hyperparameter Tuning

The SVM model was evaluated using linear, polynomial, RBF and sigmoid kernels.

The best configuration obtained through hyperparameter tuning was:

Parameter	Value
C	10
Gamma	scale
Kernel	RBF
Best CV Accuracy	0.9332
Tuning Time	120.9516 seconds

Observation:

The best SVM configuration used the RBF kernel with $C = 10$ and gamma set to scale. This configuration achieved a cross-validation accuracy of 0.9332. The tuning process required approximately 120.95 seconds. The RBF kernel provided better validation performance during tuning than the other tested configurations.

9 Baseline Logistic Regression Results

The baseline Logistic Regression model was evaluated on the test dataset before hyperparameter tuning.

Metric	Value
Accuracy	0.9294
Precision	0.9209
Recall	0.8981
F1 Score	0.9093
Training Time	0.1428 seconds

Observation:

The baseline Logistic Regression model achieved an accuracy of 92.94%. Precision was 92.09%, while recall was 89.81%. The resulting F1-score was 90.93%. The model required approximately 0.143 seconds for training, showing that Logistic Regression was relatively efficient for this dataset.

10 Tuned Logistic Regression Results

After hyperparameter tuning, Logistic Regression achieved the following results:

Metric	Value
Accuracy	0.9305
Precision	0.9211
Recall	0.9008
F1 Score	0.9109

Observation:

Hyperparameter tuning produced a small improvement in Logistic Regression performance. Accuracy increased from 92.94% to 93.05%. Precision increased from 92.09% to 92.11%, while recall increased from 89.81% to 90.08%. The F1-score also increased from 90.93% to 91.09%. Therefore, the tuned model provided slightly better overall performance than the baseline model.

11 SVM Kernel Comparison

The performance of SVM was evaluated using four different kernels.

Kernel	Accuracy	Precision	Recall	F1 Score	Training Time (s)
Linear	0.9294	0.9209	0.8981	0.9093	0.5773
Polynomial	0.7796	0.9598	0.4601	0.6220	0.5557
RBF	0.9273	0.9277	0.8843	0.9055	0.2895
Sigmoid	0.8849	0.8599	0.8457	0.8528	0.3540

11.1 SVM Prediction Time

Kernel	Prediction Time (s)
Linear	0.0253
Polynomial	0.0763
RBF	0.0864
Sigmoid	0.0573

Observation:

The linear kernel achieved an accuracy of 92.94% and an F1-score of 90.93%. The RBF kernel produced a similar accuracy of 92.73% and an F1-score of 90.55%, while requiring less training time than the linear kernel.

The polynomial kernel obtained a relatively high precision of 95.98%, but its recall was only 46.01%, resulting in a much lower F1-score of 62.20%. This indicates that the polynomial kernel missed a significant number of positive samples.

The sigmoid kernel achieved an accuracy of 88.49% and an F1-score of 85.28%. Overall, the linear kernel provided the strongest performance among the untuned SVM kernels based on accuracy and F1-score.

12 Tuned SVM Results

The best SVM configuration obtained through hyperparameter tuning was:

$$C = 10, \quad \gamma = \text{scale}, \quad \text{kernel} = \text{RBF}$$

The final test performance was:

Metric	Value
Accuracy	0.9207
Precision	0.9143
Recall	0.8815
F1 Score	0.8976
Prediction Time	0.0700 seconds

Observation:

The tuned SVM model used an RBF kernel with $C = 10$ and gamma set to scale. It achieved a test accuracy of 92.07%, precision of 91.43%, recall of 88.15% and an F1-score of 89.76%. Although its cross-validation accuracy was high, the final test accuracy was lower than that of the tuned Logistic Regression model.

13 Logistic Regression versus SVM

The final tuned models were compared using the test-set metrics.

Metric	Logistic Regression	SVM
Accuracy	0.9305	0.9207
Precision	0.9211	0.9143
Recall	0.9008	0.8815
F1 Score	0.9109	0.8976

Observation:

The tuned Logistic Regression model achieved higher test accuracy, precision, recall and F1-score than the tuned SVM model. Logistic Regression obtained an accuracy of 93.05%, whereas SVM obtained 92.07%.

The difference indicates that Logistic Regression generalized slightly better on the test set for this experiment. However, SVM achieved a higher cross-validation score during tuning, indicating that its validation performance was strong even though its final test performance was lower.

14 Five-Fold Cross-Validation

Five-fold cross-validation was performed to compare the consistency of Logistic Regression and SVM.

In each fold, four parts of the dataset were used for training and the remaining part was used for validation.

Fold	Logistic Regression	SVM
------	---------------------	-----

1	0.9131	0.9305
2	0.9326	0.9283
3	0.9283	0.9413
4	0.9239	0.9326
5	0.9239	0.9359
Average	0.9244	0.9337

Observation:

Logistic Regression achieved an average five-fold cross-validation accuracy of 92.44%, while SVM achieved a higher average accuracy of 93.37%. SVM performed better than Logistic Regression in four of the five folds, except Fold 2. The relatively small variation in the fold accuracies indicates consistent performance across the validation sets.

15 Cross-Validation Analysis

Model	Average CV Accuracy	Test Accuracy
Logistic Regression	0.9244	0.9305
SVM	0.9337	0.9207

Observation:

SVM obtained the higher average cross-validation accuracy of 93.37%, while Logistic Regression obtained 92.44%. However, the final test accuracy of Logistic Regression was higher at 93.05%, compared with 92.07% for SVM.

This difference shows that cross-validation performance and test-set performance do not necessarily produce the same ranking. Based on the final test results, Logistic Regression was the better-performing model for this experiment.

16 Performance Summary

Model	Accuracy	Precision	Recall	F1 Score
Baseline Logistic Regression	0.9294	0.9209	0.8981	0.9093
Tuned Logistic Regression	0.9305	0.9211	0.9008	0.9109
Best SVM	0.9207	0.9143	0.8815	0.8976

Observation:

Hyperparameter tuning improved the Logistic Regression model slightly across all four reported performance metrics. The tuned Logistic Regression model achieved the highest test accuracy, precision, recall and F1-score among the final models. Therefore, Logistic Regression was selected as the best-performing model based on the final test-set results.

17 Training and Tuning Time Analysis

Operation	Time (seconds)
Baseline Logistic Regression Training	0.1428
Logistic Regression Hyperparameter Tuning	289.1025
Linear SVM Training	0.5773
Polynomial SVM Training	0.5557
RBF SVM Training	0.2895
Sigmoid SVM Training	0.3540
SVM Hyperparameter Tuning	120.9516

Observation:

The baseline Logistic Regression model required only 0.1428 seconds for training. Hyperparameter tuning required considerably more time because multiple parameter combinations were evaluated using cross-validation. Logistic Regression tuning required approximately 289.10 seconds, while SVM tuning required approximately 120.95 seconds. Among the SVM kernels, the RBF kernel had the lowest training time at approximately 0.2895 seconds.

18 Discussion

The experiment compared a linear probabilistic classifier, Logistic Regression, with a margin-based classifier, SVM.

The baseline Logistic Regression model already produced strong performance with an accuracy of 92.94%. After hyperparameter tuning, its accuracy increased to 93.05%. The improvement was small but was also observed in precision, recall and F1-score.

Among the SVM kernels, the linear kernel achieved the highest accuracy of 92.94% before tuning. The RBF kernel achieved a comparable accuracy of 92.73% while requiring less training time. The polynomial kernel produced high precision but considerably lower recall, which resulted in a lower F1-score.

The tuned SVM used the RBF kernel with $C = 10$ and gamma set to scale. Its cross-validation accuracy was 93.32%, but its final test accuracy was 92.07%.

The five-fold cross-validation results showed that SVM achieved a higher average validation accuracy of 93.37% compared with 92.44% for Logistic Regression. However, Logistic Regression achieved the higher final test accuracy.

Therefore, based on the final test-set metrics, tuned Logistic Regression was selected as the best-performing model.

19 Strengths and Limitations

19.1 Logistic Regression

Strengths:

- Simple and computationally efficient.
- Easy to interpret.

- Performs well for linearly separable classification problems.
- Regularization helps control model complexity.

Limitations:

- Primarily provides a linear decision boundary.
- Performance may decrease when complex non-linear relationships exist between features.

19.2 SVM

Strengths:

- Effective for binary classification.
- Can model non-linear relationships through kernels.
- Maximizes the margin between classes.

Limitations:

- Hyperparameter tuning can be computationally expensive.
- Kernel selection affects performance significantly.
- The resulting model is less directly interpretable than Logistic Regression.

19.3 SVM Kernel-wise Accuracy

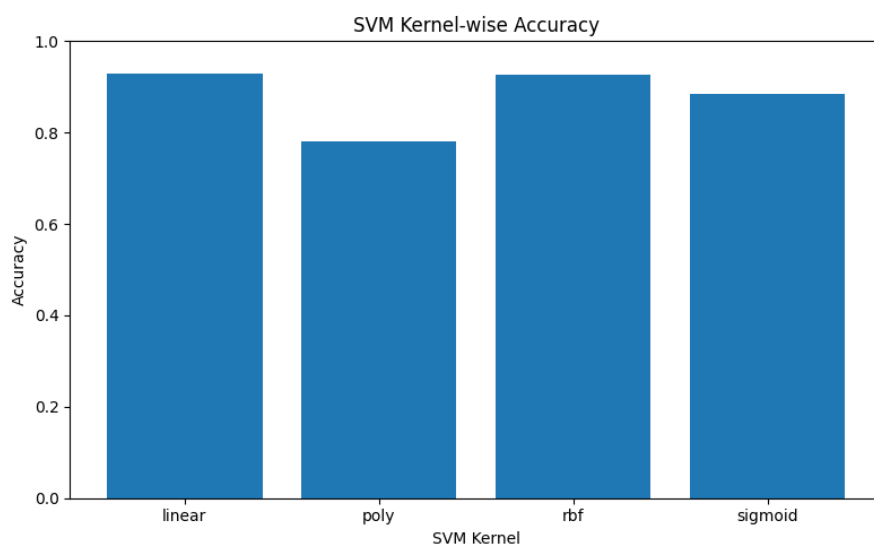


Figure 3: Accuracy comparison of different SVM kernels

Observation:

The linear kernel achieved the highest accuracy of 92.94%, followed closely by the RBF kernel with an accuracy of 92.73%. The sigmoid kernel achieved an accuracy of 88.49%, while the polynomial kernel produced the lowest accuracy of 77.96%. Therefore, the linear kernel provided the best classification accuracy among the four SVM kernels evaluated.

19.4 SVM Kernel-wise F1 Score

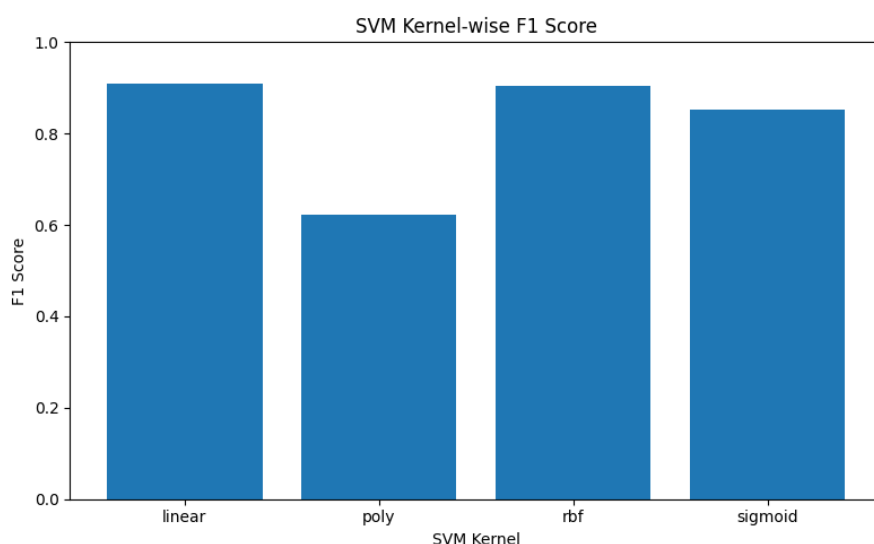


Figure 4: F1-score comparison of different SVM kernels

Observation:

The linear kernel achieved the highest F1-score of 0.9093, followed closely by the RBF kernel with an F1-score of 0.9055. The sigmoid kernel obtained an F1-score of 0.8528, whereas the polynomial kernel obtained the lowest F1-score of 0.6220. The low F1-score of the polynomial kernel is mainly associated with its low recall value of 0.4601.

19.5 Logistic Regression versus SVM

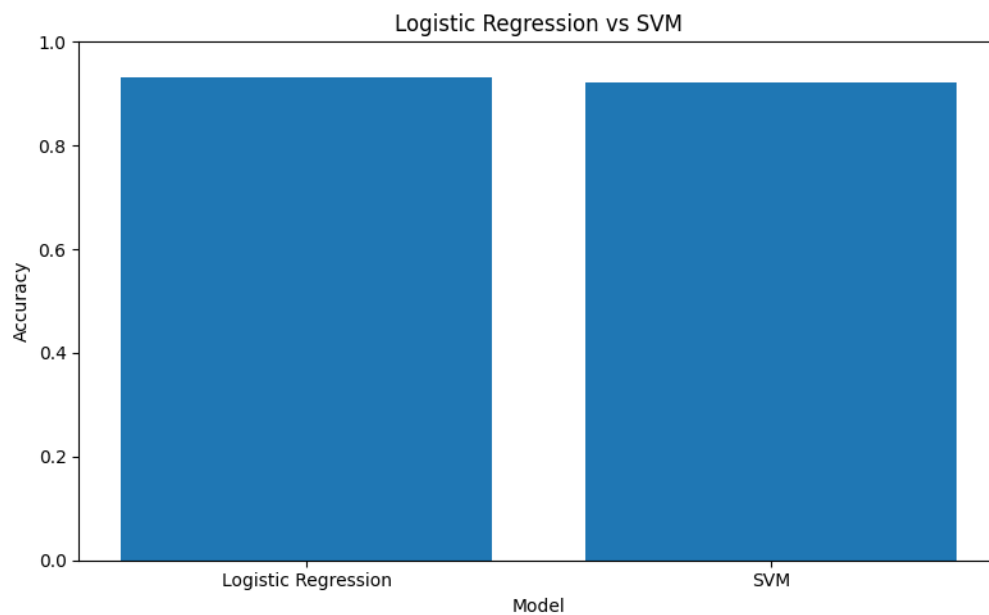


Figure 5: Performance comparison of Logistic Regression and SVM

Observation:

The tuned Logistic Regression model achieved an accuracy of 93.05%, whereas the best SVM model achieved an accuracy of 92.07%. Logistic Regression also obtained higher recall and F1-score than SVM. The precision values of both models were relatively close. Based on the final test-set results, Logistic Regression provided better overall performance than SVM.

19.6 Five-Fold Cross-Validation

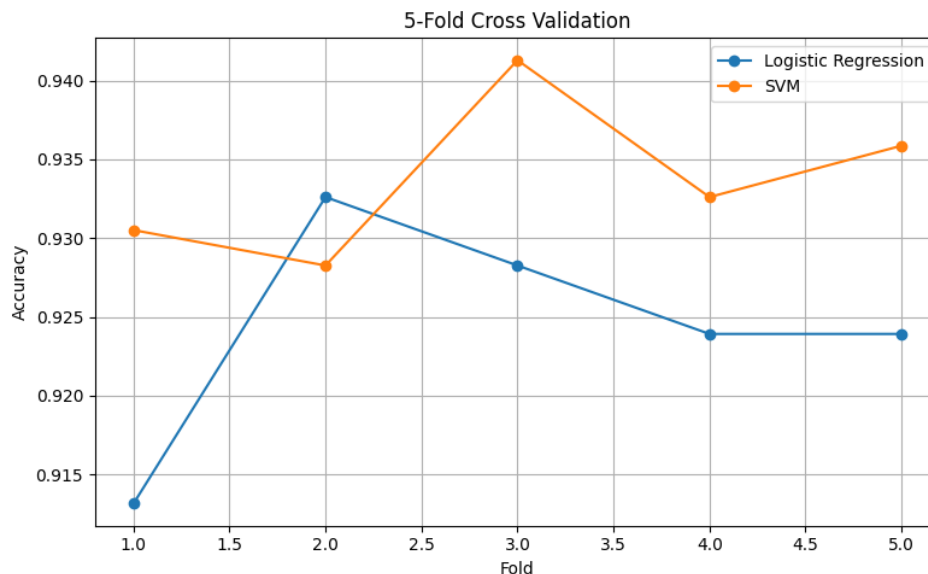


Figure 6: Five-fold cross-validation accuracy comparison

Observation:

The five-fold cross-validation results show that SVM achieved a higher average accuracy of 93.37% compared with 92.44% for Logistic Regression. SVM obtained its highest accuracy of 94.13% in Fold 3, while Logistic Regression obtained its highest accuracy of 93.26% in Fold 2. Overall, SVM showed slightly better average validation performance across the five folds.

20 Final Observation

The experiment demonstrated the application of Logistic Regression and Support Vector Machine for binary spam and ham classification.

The tuned Logistic Regression model achieved a test accuracy of **93.05%**, precision of **92.11%**, recall of **90.08%**, and F1-score of **91.09%**. Its best hyperparameters were $C = 1$, L2 regularization and the saga solver.

The best SVM configuration used an RBF kernel with $C = 10$ and gamma set to scale. It achieved a test accuracy of **92.07%**, precision of **91.43%**, recall of **88.15%**, and F1-score of **89.76%**.

Five-fold cross-validation resulted in average accuracies of **92.44%** for Logistic Regression and **93.37%** for SVM. Although SVM obtained a higher average cross-validation accuracy, Logistic Regression achieved better performance on the final test set.

Hence, based on the final test-set performance, **tuned Logistic Regression was selected as the best-performing classifier for this experiment.**

21 Conclusion

Binary classification was successfully performed using Logistic Regression and Support Vector Machine on the Spambase dataset.

Different SVM kernels were evaluated and hyperparameter tuning was performed for both models. Hyperparameter tuning improved the performance of Logistic Regression slightly. SVM showed strong cross-validation performance, particularly with the RBF kernel.

The final comparison showed that tuned Logistic Regression achieved the highest test accuracy of 93.05% and the highest F1-score of 91.09%. Therefore, Logistic Regression provided the best final test-set performance in this experiment.

The experiment also demonstrated the importance of hyperparameter selection, kernel selection and cross-validation when evaluating machine learning classification models.

22 Learning Outcomes

After completing this experiment, the following concepts were understood:

- Binary classification using Logistic Regression.
- Margin-based classification using SVM.
- Effect of L1 and L2 regularization.
- Effect of the C hyperparameter.
- Behaviour of different SVM kernels.
- Hyperparameter tuning using cross-validation.
- Evaluation using accuracy, precision, recall and F1-score.
- Five-fold cross-validation.
- Comparison of validation performance and test performance.

23 References

1. Scikit-learn Documentation, Logistic Regression.
2. Scikit-learn Documentation, Support Vector Machines.
3. Scikit-learn Documentation, Hyperparameter Optimization.
4. Spambase Dataset, UCI Machine Learning Repository.
5. Spambase Dataset, Kaggle.
6. ICS1512 Machine Learning Algorithms Laboratory, Experiment 4 Manual.